

Ascon-Keccak AEAD Algorithm

Stephan Müller

June 8, 2026

Stephan.Mueller at atsec dot com
atsec information security

Abstract

ASCON was selected as finalist in the NIST Lightweight Cryptography competition in 2023. In addition, ASCON was selected as primary choice in the final portfolio of the CAESAR competition. The ASCON specification defines among others an encryption scheme offering authenticated encryption with associated data (AEAD) which is based on a duplex mode of a sponge. With that it is the first of such algorithm selected and about to be standardized by NIST.

The sponge size is comparatively small, 320 bits, as expected for lightweight cryptography. With that, the strength of the defined AEAD algorithm is limited to 128 bits. Albeit the definition of the ASCON AEAD algorithm integrates with the associated sponge, it is mathematically not bound to exactly this sponge function. Thus, the ASCON AEAD specification can be used with a different sponge and still operate as defined by the ASCON authors.

This specification defines the ASCON-KECCAK AEAD algorithm which replaces the Ascon sponge with the KECCAK sponge, leaving the Ascon AEAD algorithm unchanged. The selected parameters for ASCON-KECCAK AEAD offer two algorithm strengths: ASCON-KECCAK 256 with a security strength compliant to the NIST security category 5. In addition, ASCON-KECCAK 512 provides an algorithm exhibiting a significantly stronger mechanism than the NIST security category 5. The selected parameters for ASCON-KECCAK 256 offer a comparable performance on 64-bit architectures to ASCON-AEAD128. The performance of ASCON-KECCAK 512 is a bit slower compared to with Ascon-AEAD128. Also, ASCON-KECCAK is significantly faster compared to AES-GCM in plain C implementations and comparable with accelerated implementations. Yet, with the KECCAK sponge size of 1600 bits, ASCON-KECCAK cannot be considered a lightweight cryptographic algorithm any more. A reference implementation of the algorithm is given with leancrypto.

Contents

1	Introduction	3
2	ASCON-KECCAK AEAD Algorithm Specification	3
2.1	Recommended Parameter Sets	4
2.2	Authenticated Encryption	5
2.2.1	Initialization	5
2.2.2	Processing of Associated Data	6
2.2.3	Processing Plaintext / Ciphertext	6
2.2.4	Finalization	6
2.3	Permutation	6
3	Security Claims	6
4	Security Analysis	8
4.1	Analysis of Authenticated Encryption	8
4.2	Analysis of the Permutation	9
5	Reference Implementation	9
5.1	Performance of ASCON-KECCAK	9

List of Tables

1	Parameters for recommended authentication encryption schemes	4
2	Security claims for recommended parameter configurations of ASCON-KECCAK	7
3	Encryption/Decryption of 1 GB Data with C implementation . .	10
4	Encryption/Decryption of 1 GB Data with accelerated implementation	12

List of Figures

1	Encryption/Decryption of 1 GB Data with C implementation . .	11
2	Encryption/Decryption of 1 GB Data with accelerated implementation	13

1 Introduction

With the selection as finalist in the NIST Lightweight Cryptography competition in 2023 and being the primary choice of the final portfolio of the CAESAR competition, ASCON is a well-recognized algorithm suite. It contains an encryption schema offering authenticated encryption with associated data (AEAD) based on a duplex mode of a sponge . It is the first of such algorithm selected and about to be standardized by NIST.

The size of the sponge comparatively small, 320 bits, which is appropriate and expected for lightweight cryptography. However, that size only allows a security strength of 128 bits for the AEAD algorithm. Even though, the ASCON AEAD algorithm definition integrates the the associated sponge, it is mathematically not bound to exactly this sponge function. Following that thought, the ASCON AEAD specification can be used with a different sponge and still operate as defined by the ASCON authors.

This specification uses the KECCAK sponge as defined [1] and integrates it into the ASCON AEAD algorithm. The selected parameters result in the following two algorithms:

- ASCON-KECCAK 256 with a security strength compliant to the NIST security category 5,
- ASCON-KECCAK 512 providing an algorithm which is significantly stronger than the NIST security category 5 following [5] appendix A.5.

A reference implementation of the algorithm is given with leancrypto.

The use of KECCAK sponge in a duplex mode was first proposed in [3] whose considerations apply to this definition as well. The KECCAK authors define an AEAD algorithm named Keyak which uses the Keccak sponge function in a similar approach as outlined in this document for ASCON-KECCAK. However, the ASCON-KECCAK specification did not consider Keyak. The reason is the following: due to ASCON being accepted by NIST as part of the lightweight cryptography competition, an AEAD algorithm specification using a sponge function is considered a finalist during the public competition. Thus, this accepted AEAD specification is used and simply the sponge function is replaced with appropriate parameter sets with the expectation that the security analysis of ASCON is still applicable.

2 ASCON-KECCAK AEAD Algorithm Specification

The ASCON specification provided with [6] contains in chapter 4 the algorithm definition for the AEAD encryption and decryption operation. This algorithm specification outlines how the sponge state is initialized with key, IV and a nonce, how the associated data is authenticated, how plaintext data is transformed into ciphertext and vice versa, and how the authentication tag is obtained.

The ASCON permutation operation forming the basis for the sponge is defined in [6] chapter 3.

Name	Key (Bits)	Nonce (Bits)	Tag (Bits)	Data Block (Bits)	Rounds p^a	Rounds p^b
ASCON- KECCAK 256	256	128 to 256	128 to 256	1,088	24	24
ASCON- KECCAK 512	512	128 to 512	128 to 512	576	24	24

Table 1: Parameters for recommended authentication encryption schemes

Finally, the parameter set for ASCON-AEAD128 is defined in [6] section 4.1 which defines the sizes of the key, nonce and authentication tag along with the ASCON sponge round counts.

The following sections define the ASCON-KECCAK AEAD algorithm based on the same structure as the original ASCON specification [4].

2.1 Recommended Parameter Sets

The authenticated encryption uses the parameter sets defined in Table 1. It specifies the same parameters as the original ASCON definition in [6] section 4.1.

The basic considerations for the selected parameter sets are the following:

- The KECCAK permutation state size and the round counts are equal to the definition of KECCAK-p[1600,24] specified in [1], i.e. the definition used for SHA-3 and SHAKE.
- ASCON-KECCAK 256 segments the KECCAK sponge state size into the rate / capacity equal to the parameters used for SHA3-256 defined in [1], i.e. capacity of 512 bits and rate 1,088 bits.
- ASCON-KECCAK 512 segments the KECCAK sponge state size into the rate / capacity equal to the parameters used for SHA3-512 defined in [1], i.e. capacity of 1,024 bits and rate 576 bits.

From the basic considerations for the parameter sets, the following ASCON-KECCAK parameter set rationale applies:

- The key size is defined to be equal to half of the selected capacity of the sponge.
- The tag size is at least 128 bits as required by the ASCON specification, but is allowed to be larger up to the key size. This is based on the fact that the tag is obtained from the data block of the sponge state which is modified by the key as the last ASCON AEAD encryption / decryption step. Thus, the tag shall not be larger than the key size.
- The data block size is equal to the rate of the selected sponge.

2.2 Authenticated Encryption

The authenticated encryption is defined to be basically equal to the ASCON encryption specification given in [6] section 4.1.1 for encryption and section 4.1.2 for decryption with adjustments only due to KECCAK specifics of the state size and the increased size of the tag.

2.2.1 Initialization

The initialization vector is defined based on the specification given in [6] section 4.1.1 algorithm 3 / section 4.1.1 algorithm 4. Considering that KECCAK also operates on 64-bit words, the IV size is equally defined as 64 bit value using the parameter set defined in Table 1.

To provide proper domain separation for the applied tag size, the IV construction is extended to include the size of the tag value. Therefore, the IV is defined as:

$$IV_{k,r,a,b,t} = k || r || a || b || t$$

with:

- k referencing the key size in bytes represented by an 8-bit integer,
- r referencing the rate in bytes represented by an 8-bit integer,
- a referencing the initialization and finalization round number represented by an 8-bit integer,
- b referencing the intermediate round number represented by an 8-bit integer,
- t referencing the tag size in bytes represented by an 8-bit integer,

The applied IVs are as follows for the example of the minimum and maximum tag sizes – when using other tag sizes, the fifth byte in the IV must be set accordingly:

- ASCON-KECCAK 256 with tag size 128 bits: 0x2088181810000000
- ASCON-KECCAK 256 with tag size 256 bits: 0x2088181820000000
- ASCON-KECCAK 512 with tag size 128 bits: 0x4048181810000000
- ASCON-KECCAK 512 with tag size 512 bits: 0x4048181840000000

Considering that the KECCAK sponge size is 1,600 bits compared to the ASCON sponge size of 320 bits, and the requirement that the key is inserted at the end of the sponge state, the initialization is performed with the following steps:

$$S \leftarrow IV_{k,r,a,b,t} || K || N$$

This is followed by a second insertion of the key after a sponge operation:

$$S \leftarrow p^a(S) \oplus (0^{1600-k} || K)$$

Thus, the initialization is identical to the specification of [6] section 4.1.1 algorithm 3 / section 4.1.1 algorithm 4 after applying the KECCAK sponge size.

2.2.2 Processing of Associated Data

The processing of the associated data is fully identical to the specification of [6] section 4.1.1 algorithm 3 / section 4.1.1 algorithm 4 with the only difference that the domain separation constant inserted into the last bit of the sponge state must consider the KECCAK state size:

$$S \leftarrow S \oplus (0^{1599} || 1)$$

2.2.3 Processing Plaintext / Ciphertext

The processing of the plaintext / ciphertext is fully identical to the specification of [6] section 4.1.1 algorithm 3 / section 4.1.1 algorithm 4.

2.2.4 Finalization

The finalization is fully identical to the specification of [6] section 4.1.1 algorithm 3 / section 4.1.1 algorithm 4 with the exception that the tag is defined as follows depending on the chosen tag size t :

$$S \leftarrow p^a(S \oplus (0^r || K || 0^{c-k}))$$

$$T \leftarrow \lceil S \rceil^t \oplus \lceil K \rceil^t$$

with

$$128 \leq t \leq k$$

2.3 Permutation

As already indicated in the preceding sections, the KECCAK permutation as specified in [1] section 5.2 is applied to replace the ASCON permutation operation. Specifically, the permutation of KECCAK-p[1600,24] is applied, which is also the basis for SHA-3 and SHAKE.

3 Security Claims

The ASCON-KECCAK 256 algorithm provides 256-bit security strength, and ASCON-KECCAK 512 provides 512-bit security strength for the protection of the confidentiality of the data. The security strength of the integrity depends

Requirement	ASCON-KECCAK 256 (Bits)	ASCON-KECCAK 512 (Bits)
Confidentiality of plaintext	256	512
Integrity of plaintext	128 to 256	128 to 512
Integrity of associated data	128 to 256	128 to 512
Integrity of public message number	128 to 256	128 to 512

Table 2: Security claims for recommended parameter configurations of ASCON-KECCAK

on the selected tag length and can reach 256-bit security strength for ASCON-KECCAK 256 and 512-bit security strength for ASCON-KECCAK 512 if the largest tag size is selected.

The number of processed plaintext and associated data blocks protected by the encryption algorithm is limited to the same number as specified in [4] section 3.1: 2^{64} blocks. This translates into $2^{71} + 2^{67}$ bytes for ASCON-KECCAK 256 and $2^{70} + 2^{67}$ bytes for ASCON-KECCAK 512. As stated in [4] section 3.1: “In order to fulfill the security claims stated in Table 2, implementations must ensure that the nonce (public message number) is never repeated for two encryptions under the same key, and that decrypted plaintexts are only released after successful verification of the final tag. The difference between the family members is in their robustness against other adversaries beyond the classical security claim and is discussed in the following.”

The security strength of the confidentiality depends on the capacity of the KECCAK sponge where the attack complexity is $2^{c/2}$ which implies that the robustness of ASCON-KECCAK 256 is 256 bits ($c = 512$), and ASCON-KECCAK 512 is 512 bits ($c = 1024$) based on the assessment given in [4] section 3.1. The security strength claims are supported by the numbers specified in [1] Appendix A.1 for SHA3-256 and SHA3-512 considering that the KECCAK sponge is operated identically to SHA3-256 and SHA3-512, respectively.

According to [4] section 3.1, the same holds true for key recovery attacks.

The security strength of the integrity of the plaintext and associated data depends on the size of the tag which is allowed to be between 128 bits and 256 or 512 bits, respectively.

Considering Grover’s algorithm, the security strength against a quantum adversary depends on the key size and the capacity of the KECCAK sponge. It is therefore defined as $\min(\text{key size}, 2^{c/4})$. Therefore, ASCON-KECCAK 256 has a security strength of 128 bits and ASCON-KECCAK 512 has a security strength of 256 bits against quantum adversaries. However, as outlined in [5] Appendix A.5, this naïve calculation only considers the quadratic quantum speedup but ignores the “long-running serial computation, which is difficult to implement in practice. In a realistic attack, one has to run many smaller instances of the algorithm in parallel, which makes the quantum speedup less dramatic.” Thus,

considering the table in [5] Appendix A.5, the security strength of ASCON-KECCAK is therefore defined as:

- ASCON-KECCAK 256 complies with NIST security category 5.
- ASCON-KECCAK 512 is significantly stronger than NIST security category 5. Considering it operates the Keccak sponge equivalent to SHA3-512 the security of ASCON-KECCAK 512 is compliant to the security strength of SHA3-512 as defined in table 5 in [5] Appendix A.5 by requiring 2^{274} classical gates for an optimal key recovery attack. This indicates a significantly stronger algorithm than the NIST security category 5.

As outlined in [4] section 3.1, except for the single-use requirement, there are no constraints on the choice of the nonce (public message number); in particular, it is possible to use a simple counter.

4 Security Analysis

The entire specification of ASCON-KECCAK is based on the identical reuse of ASCON AEAD with the difference that the sponge is replaced. The following subsections provide the security analysis of the relevant components following [4] chapter 6.

4.1 Analysis of Authenticated Encryption

The security analysis given in [4] section 6.2.2 applies in full considering that the ASCON authenticated encryption is applied. The deviations from the ASCON specification have the following effect:

- Initialization: The algorithm difference places the key into the last bits of the sponge state. As the ASCON sponge size is different than the KECCAK sponge size, the mathematical representation of the operation differs. Yet, this mathematical difference has no bearing on the security posture of the algorithm.
- Finalization: The tag size differs to allow larger tag sizes at most equal to the key size. As the key size is also strictly smaller than the capacity of the KECCAK sponge, and the largest allowed tag size are equal to the key size, the specification is identical to the ASCON concepts. Thus, security analysis results from ASCON applies in full.
- Endianness: The difference in endianness between the ASCON sponge and the KECCAK sponge mentioned in chapter 5 is considered to have no impact on the security posture as still all data is processed in the same manner.

4.2 Analysis of the Permutation

As the KECCAK permutation is operated identical to the SHA3-256 and SHA3-512 hashing operation, the security analysis provided by [2] chapter 4 is applicable based on the claims in [1] Appendix A. Thus, this security analysis applies to the specified ASCON-KECCAK algorithms in full.

5 Reference Implementation

A reference implementation of ASCON-KECCAK is provided with leancrypto. To demonstrate also that its implementation matches the aforementioned specification, the following considerations are applied:

- The library implements ASCON-AEAD128 which is demonstrated to match the implementation given in [6] by obtaining NIST CAVP certificates such as A7538..
- The ASCON-AEAD128 implementation is such that the ASCON AEAD operation is separated from the sponge implementation.
- The ASCON-KECCAK 256 and 512 implementations are provided by simply replacing the ASCON permutation and applying the KECCAK sponge size for the initialization operation. Thus, the ASCON AEAD code is shared for both ASCON-128 and ASCON-KECCAK.
- The KECCAK sponge implementation is also used for the SHA-3 algorithm family which is demonstrated to correctly generate the respective message digests using the NIST reference implementation provided with the ACVP service including obtaining official certificates, such as A4850.

5.1 Performance of ASCON-KECCAK

Based on the reference implementation, the following performance data is obtained. The table 3 along with the visualization in figure 1 using a logarithm scale show the performance data with the C implementation of both sponges, KECCAK and ASCON. The used constant-time C implementation of the AES block operation listed in the following table is slower compared to an AES implementation with S-Boxes where the implementation with S-Boxes require only about half the time. Yet, this constant-time AES implementation is used in production code to mitigate side-channels around the key and the key schedule. The other algorithm implementations also are implemented such that side-channels circling around the key are mitigated. Such effort, however, is significantly simpler compared to AES, as the sponge operation processing the key does not require the use of lookup tables and similar which are prone to side-channel attacks.

Hardware	Word size	ASCON-AEAD128	ASCON-KECCAK 256	ASCON-KECCAK 512	AES-GCM	ChaCha20-Poly1305
ARMv8 Cortex-A72	64-bit	21.47s	19.75s	32.60s	397.04s	18.19s
ARMv8 Cortex-A76	64-bit	10.54s	10.15s	17.03s	219.66s	10.02s
Apple M3 Max	64-bit	3.78s	2.92s	4.90s	111.96s	3.24s
Apple M4 Max	64-bit	3.23s	3.45s	6.19s	103.97s	2.80s
ARMv7 Cortex-A7	32-bit	11.66s	20.49s	36.98s	144.80s	6.57s
ARMv7 Cortex-A8	32-bit	9.47s	15.84s	28.01s	120.70s	6.70s
Intel 10th Gen Core i7 10810U	64-bit	8.60s	10.78s	18.12s	176.07s	5.89s
Intel 11th Gen Core i7 1165G7	64-bit	7.32s	6.90s	11.78s	164.38s	4.77s
Intel 12th Gen Core i7 1280P	64-bit	5.40s	5.65s	9.75s	133.83s	3.93s
Intel 14th Gen Ultra 7 155H	64-bit	5.68s	6.65s	11.13s	128.94s	4.17s
RISC-V Spacemit K1 X60	64-bit	37.61s	41.81s	68.30s	746.69s	31.02s

Table 3: Encryption/Decryption of 1 GB Data with C implementation

The performance may look a bit surprising at first considering that ASCON-KECCAK 256 is sometimes faster on 64-bit architectures¹ than the fastest ASCON AEAD algorithm and knowing that the KECCAK sponge (a) is more complex and (b) performed with a significant larger round count. But the factor of the processed data block size plays the leading role. For ASCON-KECCAK it is defined to be equal to the rate which is significantly larger than for ASCON AEAD. Thus the data is processed by ASCON-KECCAK in much larger data blocks than for ASCON AEAD.

Furthermore, the performance measurements show that ASCON-KECCAK is significantly faster by more than 10 times up to 30 times compared to AES-GCM (and about 5 to 15 times to an more insecure AES implementation with an S-

¹The 32-bit ARM Cortex-A7 shows that ASCON-KECCAK is generally slower than ASCON AEAD. It is assumed that this is due to the fact that this CPU only offers 16 64-bit registers compared to ARM Cortex-A8 which offers 32 64-bit registers. Thus, the KECCAK state of 25 64-bit integers will not fit completely into the existing numbers of registers, requiring the compiler to either reload existing registers or to split the processing of some 64-bit integers into sets of two 32-bit operations.

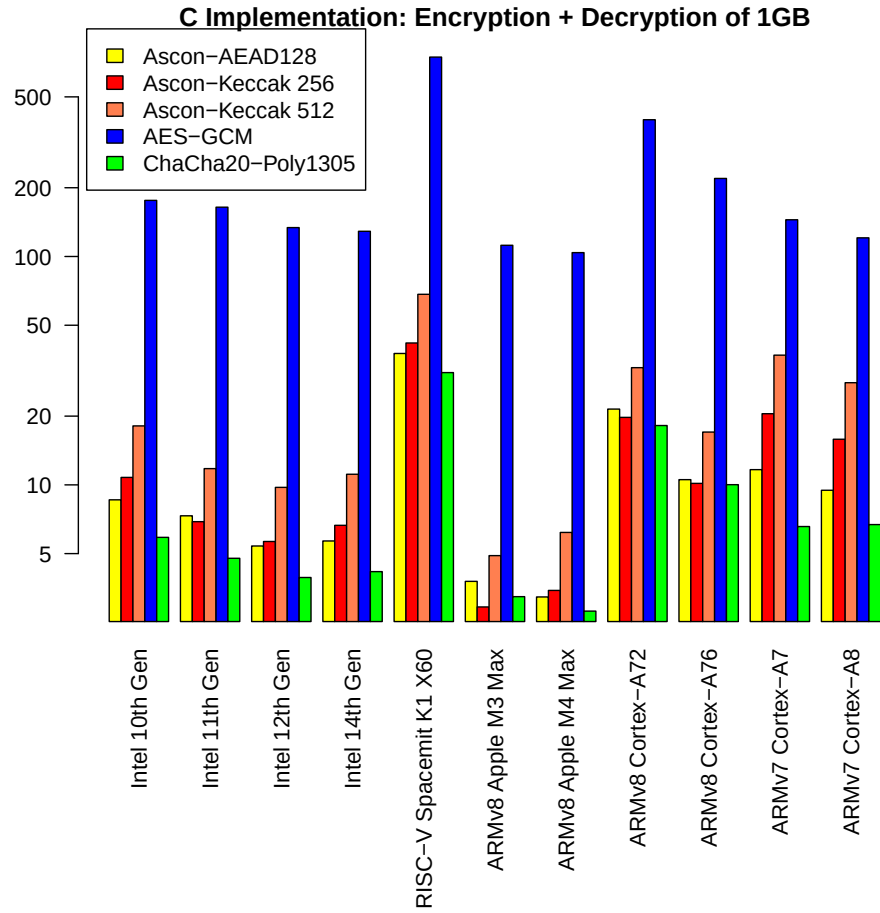


Figure 1: Encryption/Decryption of 1 GB Data with C implementation

Box lookup). Comparing ASCON-KECCAK with ChaCha20 the performance data shows that on average ASCON-KECCAK is slower by about 20%.

In addition, a similar comparison with accelerated algorithm implementations is visualized in table 4 and figure 2.

Hardware	Word size	ASCON-AEAD128	ASCON-KECCAK 256	ASCON-KECCAK 512	AES-GCM	ChaCha20-Poly1305
ARMv8 Cortex-A72	64-bit	21.40s	18.03s	30.65s	N/A	13.17s
ARMv8 Cortex-A76	64-bit	10.55s	8.84s	15.18s	5.65s	7.07s
Apple M3 Max	64-bit	3.80s	2.53s	4.35s	1.94s	1.76s
Apple M4 Max	64-bit	3.23s	2.08s	3.74s	1.55s	1.37s
ARMv7 Cortex-A7	32-bit	8.09s	20.50s	36.96s	N/A	4.34s
ARMv7 Cortex-A8	32-bit	7.88s	5.90s	9.99s	N/A	4.71s
Intel 10th Gen Core i7 10810U	64-bit	N/A	6.11s	11.42s	5.91s	2.79s
Intel 11th Gen Core i7 1165G7	64-bit	6.46s	3.60s	6.38s	4.58s	1.82s
Intel 12th Gen Core i7 1280P	64-bit	N/A	4.80s	8.68s	4.81s	1.76s
Intel 14th Gen Ultra 7 155H	64-bit	N/A	5.11s	8.76s	4.27s	2.14s
RISC-V Spacemit K1 X60	64-bit	N/A	38.70s	55.06s	75.91s	20.76s

Table 4: Encryption/Decryption of 1 GB Data with accelerated implementation

Table 4 contains “N/A” if no acceleration is available for the given algorithm.

For the accelerated implementations, ASCON-KECCAK is sometimes a bit slower, but also occasionally a bit faster compared to AES-GCM. The reader should for the interpretation of the numbers, however, consider that AES-GCM uses the high-performance AES CPU instructions where available (Intel AES-NI/PCLMULQDQ or ARM CE/PMULL) compared to ASCON-KECCAK mostly “only” using vector mathematical operations for the sponge (Intel AVX2, AVX-512, ARM Neon).

The acceleration implementations are as follows:

- Intel 10th Gen

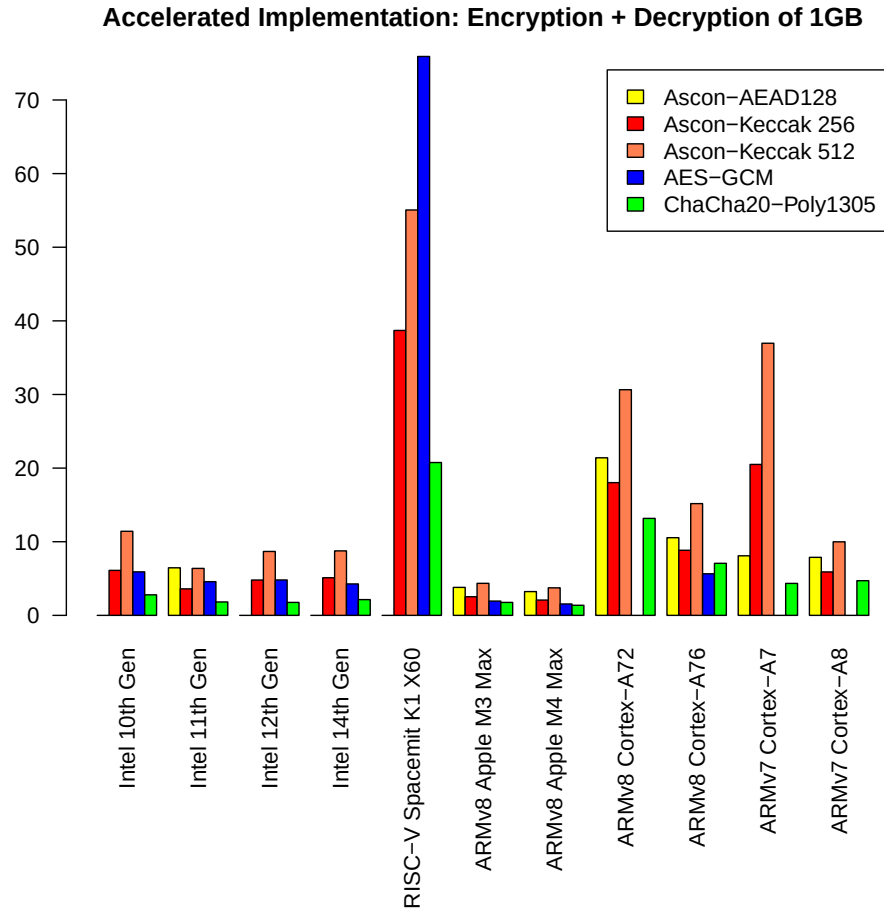


Figure 2: Encryption/Decryption of 1 GB Data with accelerated implementation

- Ascon: N/A
- Keccak: AVX2
- AES: AES-NI
- GCM: PCLMULQDQ
- ChaCha20: AVX2
- Poly1305: N/A
- Intel 11th Gen
 - Ascon: AVX512
 - Keccak: AVX512
 - AES: AES-NI
 - GCM: PCLMULQDQ
 - ChaCha20: AVX512
 - Poly1305: N/A
- Intel 12th Gen
 - Ascon: N/A
 - Keccak: AVX2
 - AES: AES-NI
 - GCM: PCLMULQDQ
 - ChaCha20: AVX2
 - Poly1305: N/A
- Intel 14th Gen
 - Ascon: N/A
 - Keccak: AVX2
 - AES: AES-NI
 - GCM: PCLMULQDQ
 - ChaCha20: AVX2
 - Poly1305: N/A
- RISC-V Spacemit K1 X60
 - Ascon: N/A
 - Keccak: RISC-V Zbb
 - AES: Assembler
 - GCM: RISC-V Zbb
 - ChaCha20: RISC-V Zbb

- Poly1305: N/A
- ARMv8 Apple M3 Max
 - Ascon: Neon
 - Keccak: ARM-CE
 - AES: ARM-CE
 - GCM: PMULL
 - ChaCha20: Neon
 - Poly1305: N/A
- ARMv8 Apple M4 Max
 - Ascon: Neon
 - Keccak: ARM-CE
 - AES: ARM-CE
 - GCM: PMULL
 - ChaCha20: Neon
 - Poly1305: N/A
- ARMv8 Cortex-A72
 - Ascon: Neon
 - Keccak: Assembler
 - AES: N/A
 - GCM: N/A
 - ChaCha20: Neon
 - Poly1305: N/A
- ARMv8 Cortex-A76
 - Ascon: Neon
 - Keccak: Assembler
 - AES: ARM-CE
 - GCM: PMULL
 - ChaCha20: Neon
 - Poly1305: N/A
- ARMv7 Cortex-A7
 - Ascon: Neon
 - Keccak: Neon

- AES: N/A
- GCM: N/A
- ChaCha20: Neon
- Poly1305: N/A
- ARMv7 Cortex-A8
 - Ascon: Neon
 - Keccak: Neon
 - AES: N/A
 - GCM: N/A
 - ChaCha20: Neon
 - Poly1305: N/A

Acknowledgements

The author thanks Caroline Holz auf der Heide, and Joachim Vandersmissen for comments on earlier drafts of the paper.

References

- [1] *FIPS PUB 202 SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. NIST, August 2015. <https://doi.org/10.6028/NIST.FIPS.202>.
- [2] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. *The KECCAK reference*. January, 2011. <http://keccak.noekeon.org/Keccak-reference-3.0.pdf>.
- [3] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Duplexing the sponge: Single-pass authenticated encryption and other applications. In Ali Miri and Serge Vaudenay, editors, *Selected Areas in Cryptography - 18th International Workshop, SAC 2011, Toronto, ON, Canada, August 11-12, 2011, Revised Selected Papers*, volume 7118 of *Lecture Notes in Computer Science*, pages 320–337. Springer, 2011.
- [4] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläpfer. *Ascon Submission to NIST*. May 31, 2021. <https://ascon.iaik.tugraz.at/files/asconv12-nist.pdf>.
- [5] NIST. Submission requirements and evaluation criteria for the post-quantum cryptography standardization process. Official Call for Proposals, National Institute for Standards and Technology, December 2016.
- [6] Meltem Sönmez Turan, Kerry A. McKay, Jinkeon Kang, and John Kelsey. *NIST SP 800-232 Ascon-Based Lightweight Cryptography Standards for Constrained Devices*. August, 2025. <https://doi.org/10.6028/NIST.SP.800-232>.